

University of Engineering and Technology, Lahore

Department of Computer Science

Course: CSC-203 Computer Networks

Complex Computing Problem (CCP)

Project Title:

Real-time Collaborative Network Monitoring and Alert Dashboard

Submitted By:

- Name 1 (Roll No.)
- Name 2 (Roll No.)
- Name 3 (Roll No.)

Submitted To: [Professor Name]

Date: June 07, 2026

Table of Contents

Section	Page
1. Introduction	3
2. Problem Statement	4
3. System Architecture	5
4. Protocol Design	7
5. Implementation Details	8
6. Features List	11
7. Challenges & Solutions	12
8. Testing & Results	13
9. CLO Alignment	16
10. Conclusion	17
References	18

1. Introduction

Network monitoring is an important part of computer networks because administrators must know when a router, server, service, or link is behaving abnormally. In a real Network Operations Center, tools such as Nagios and PRTG collect events, show active devices, and alert operators when a problem needs immediate action.

This project implements the same basic idea on a small scale using Python socket programming. Multiple clients connect to one central server over TCP. The clients can broadcast status updates, send emergency alerts, join category-based monitoring rooms, exchange private incident messages, and share log files or screenshots.

The main problem solved by this system is coordination. When several users are watching simulated network events, they need a common dashboard where important updates arrive in real time. This makes the project useful for understanding application-layer message design, transport-layer reliability, concurrency, and event-driven communication.

- Real world use: network labs, help desk rooms, NOC dashboards, and incident response teams.
- Main networking concept: client-server communication over TCP.
- Main programming concept: threads for handling many clients at the same time.

2. Problem Statement

The assignment requires students to design and develop a real-time Collaborative Network Monitoring and Alert Dashboard using socket programming. The system must allow multiple connected clients, more than three devices, to monitor, share, and respond to simulated network events over a LAN or Wi-Fi network.

Required functions include multi-client connectivity, real-time alert broadcasting, active client node display with hostnames and IP addresses, event log sharing, group monitoring rooms, private direct communication, and simulated voice or text emergency alerts.

In our implementation, the above requirements are handled by a threaded Python TCP server and two client interfaces: a GUI dashboard for normal operation and a CLI client for quick demonstrations. The system can be demonstrated on one laptop by opening one server terminal and three client terminals.

3. System Architecture

The architecture follows a client-server model. The server is the central monitoring point. All clients connect to it and send JSON messages over TCP. The server keeps the active node list, routes room messages, forwards private messages, and broadcasts emergency alerts.

Client-Server Architecture

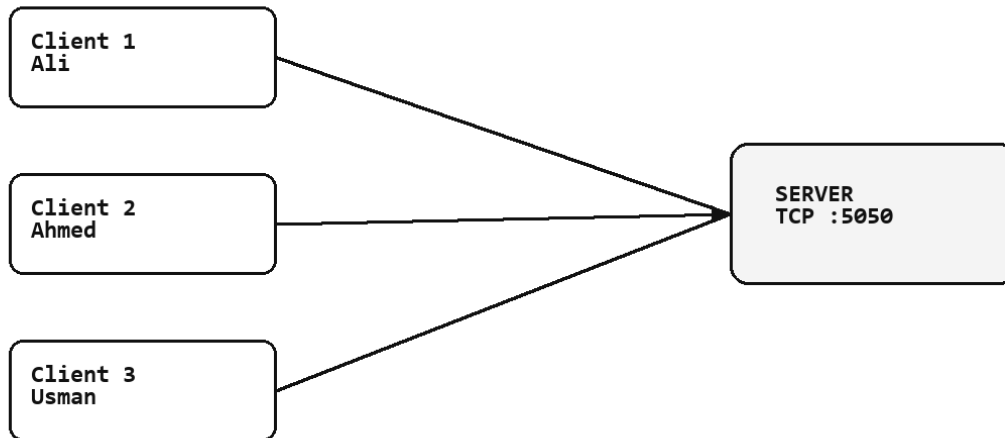


Figure 1: Client-server architecture used in the project

Even when all terminals run on the same PC, the design still behaves like a LAN model because every client opens its own socket connection to the server. On a real Wi-Fi network, other laptops can connect by using the server laptop IP address instead of 127.0.0.1.

Node Topology over LAN / Wi-Fi

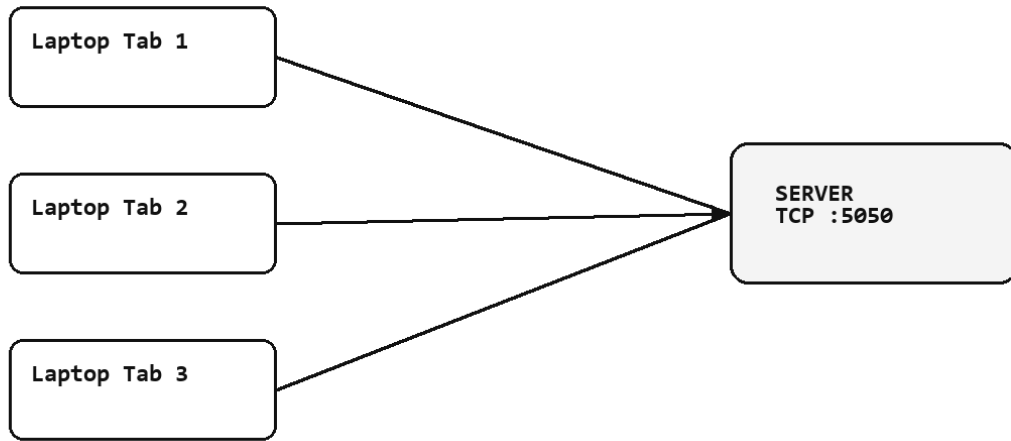


Figure 2: Node topology for three clients connected to one server

4. Protocol Design

TCP was selected instead of UDP because the dashboard needs reliable delivery. Alerts, private messages, and shared files should not be silently lost. TCP also preserves byte stream ordering, which makes the newline-delimited JSON application protocol simple to implement.

- Application layer: JSON messages such as hello, alert, private, room_message, file, nodes_request, ping and pong.
- Transport layer: TCP sockets with server port 5050.
- Message framing: every JSON object is encoded in UTF-8 and terminated by a newline.
- Reliability reason: TCP provides retransmission, ordered delivery, and connection state.

Example message format: `{"type":"alert", "text":"CPU usage crossed 90 percent"}`. For files, the client reads the file bytes and sends a Base64 encoded payload with filename and size metadata.

5. Implementation Details

5.1 Server Side

The server opens a TCP socket, binds it to port 5050, and listens for incoming client connections. Every accepted client is handled in a separate thread. A shared dictionary stores client names, hostnames, IP addresses, connected time, and subscribed rooms. A lock is used whenever shared state is read or updated.

5.2 Client Side

The GUI client uses Tkinter. It has controls for connecting to the server, sending broadcast messages, emergency alerts, joining rooms, sending private messages, sharing files, and viewing active nodes. The CLI client uses command-style input so the demo can be shown quickly in terminals.

5.3 Group Rooms

The rooms are CPU, Bandwidth, and Security. A client can join or leave any room. Room messages are only delivered to clients subscribed to that room, which is similar to alert categories in real monitoring tools.

5.4 Private Messaging

For private communication, the sender provides the target client name. The server searches the active client table and forwards the private payload only to the target and the sender for confirmation. Other clients do not receive that message.

5.5 File Sharing

The file-sharing feature supports text logs, screenshots, and short videos. For the demo, small files are encoded with Base64 and sent through the same TCP connection. The server stores a copy and broadcasts the file metadata and payload to connected clients.

5.6 Emergency Alerts

Emergency alerts use a separate message type. In the GUI client, the local notification bell is triggered and the log line is highlighted. In CLI mode, the alert is printed with an EMERGENCY label so it is easy to see during presentation.

5.7 Code Structure

File	Purpose
server.py	Accepts TCP clients, maintains active node state, manages rooms, private chat, alerts, and file forwarding.
client_gui.py	Tkinter dashboard for live monitoring, alerts, nodes, private chat, rooms, latency, and file

	sharing.
client_cli.py	Terminal client for fast presentation and command-based testing.
protocol.py	Shared JSON send/receive helpers and newline framing logic.
test_integration.py	Automated test covering the required communication features.

6. Features List

- Multi-client connectivity for three or more clients.
- Real-time broadcasting of status updates and alert notifications.
- Active nodes display with IP address and hostname.
- File and log sharing between clients.
- Group monitoring rooms for CPU, Bandwidth, and Security alerts.
- Private direct communication between two selected clients.
- Emergency alert system with highlighted notification.
- Optional feature: GUI dashboard and latency ping metric.

7. Challenges & Solutions

Challenge	Solution Applied
Multiple clients at the same time	A separate thread is created for every client connection.
Shared active node state	A lock protects the dictionary containing connected clients and room subscriptions.
Room subscription management	A set of room names is stored with every client record.
Client disconnect handling	Socket errors and empty reads are handled with try/except and cleanup.
File transfer	Files are transferred as Base64 payloads and stored by the server.

8. Testing & Results

Testing was performed on one PC using separate terminals for the server and clients. The automated integration test also verified the main flows: active nodes, alert broadcasting, room message delivery, private chat, file sharing, and ping response.



Screenshot 1: Three clients connected and visible in the active node list

Screenshot 2: Real-time Emergency Alert Broadcast

Ali	Ahmed	Usman
<pre>/emergency Core router latency critical [EMERGENCY] Ali: Core router latency critical</pre>	<pre>[EMERGENCY] Ali: Core router latency critical [SYSTEM] Audible notification triggered</pre>	<pre>[EMERGENCY] Ali: Core router latency critical [METRIC] received instantly</pre>

Screenshot 2: Emergency alert received by all connected clients

Screenshot 3: Private Direct Communication

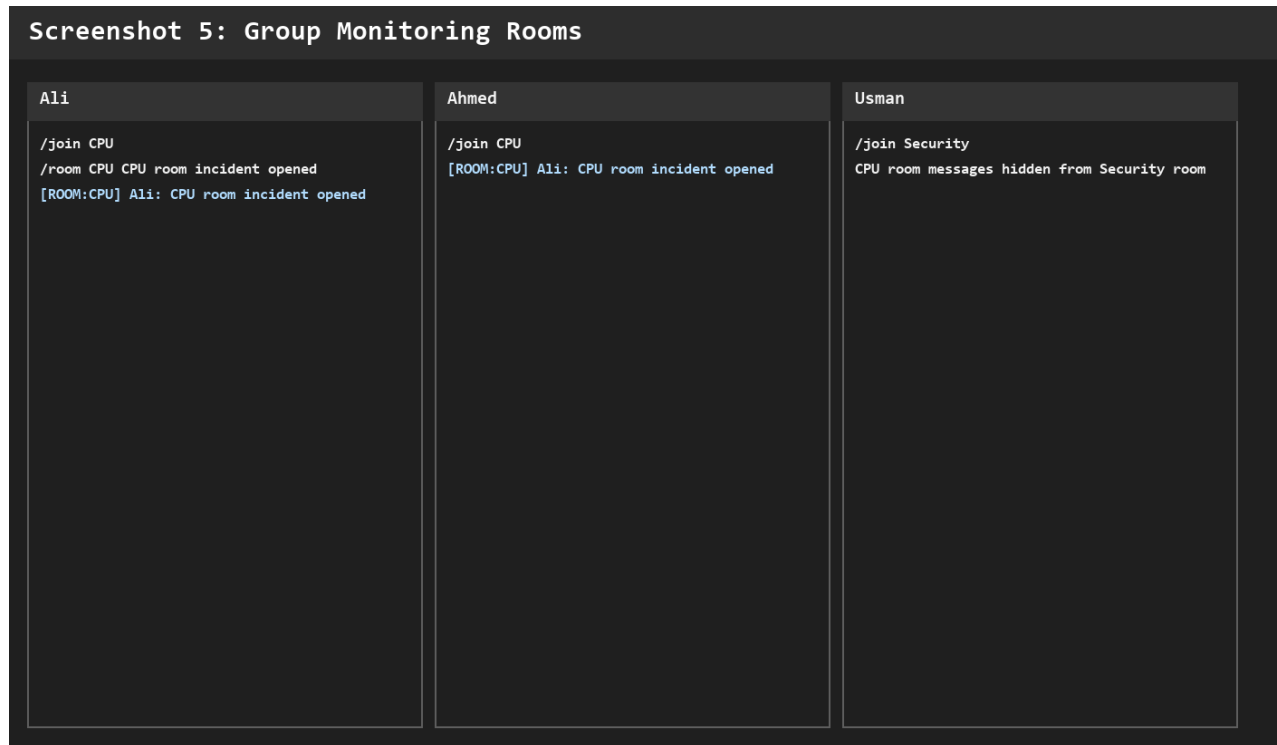
Usman	Ahmed	Ali
<pre>/pm Ahmed Please verify firewall logs [PRIVATE] Usman -> Ahmed: Please verify firewall logs</pre>	<pre>[PRIVATE] Usman -> Ahmed: Please verify firewall logs [PRIVATE] Usman Checking IDS events now</pre>	<pre>[PRIVATE] Usman -> Ahmed: Please verify firewall logs Broadcast log remains clean</pre>

Screenshot 3: Private message delivered only to selected users

Screenshot 4: File / Log Sharing

Ali	Ahmed	Server
<pre>/file sample_logs/cpu_fault_log.txt [FILE] Ali shared cpu_fault_log.txt (209 bytes)</pre>	<pre>[FILE] Ali shared cpu_fault_log.txt (209 bytes) Saved event log for incident notes</pre>	<pre>[FILE] Ali shared cpu_fault_log.txt Stored in server_received folder</pre>

Screenshot 4: File sharing of a sample CPU fault log



Screenshot 5: CPU group room message received only by subscribed clients

Performance observations: on local testing, the ping response was normally below a few milliseconds because all clients were on the same PC. The threaded server handled three simultaneous clients smoothly. Since TCP is used, packet loss is handled by the transport layer through retransmission.

9. CLO Alignment

CLO 3 is about analyzing network protocols based on application and transport layer performance requirements. This project aligns with CLO 3 because it required a protocol choice, application message format design, and concurrency decisions.

- TCP was selected after comparing reliability needs with UDP behavior.
- Application-layer JSON messages were designed for alerts, rooms, private chat, file sharing, and node discovery.
- Threading was used to support concurrent clients without blocking the whole server.
- The system demonstrates interdependent components: server, clients, rooms, files, alerts, and active node display.

10. Conclusion

The project successfully implements a real-time collaborative network monitoring and alert dashboard using Python sockets and threading. It supports multi-client connectivity, active node display, broadcast alerts, group rooms, private chat, file/log sharing, and emergency notifications.

The system is suitable for classroom demonstration because it can run completely on one laptop using multiple terminals. Future improvements can include TLS encryption, persistent chat history, user authentication, larger chunk-based file transfer, and a web dashboard.

References

- [1] Python Socket Programming Documentation
- [2] Python Threading Documentation
- [3] Behrouz A. Forouzan, Data Communications and Networking
- [4] Nagios Documentation
- [5] PRTG Network Monitor Documentation